

차세대 산업용 IoT 및 실시간 SCADA 데이터 파이프라인 아키텍처: 분산 메시징, 결측치 보간, 스트림 처리 및 머신러닝 기반 이상 탐지 통합 체계

분산 산업용 IoT 파이프라인의 데이터 흐름 및 타당성 평가

현대 제조 공정 및 스마트 팩토리 환경에서는 수만 개의 센서 노드와 프로그래머블 로직 컨트롤러(PLC)에서 방출되는 고빈도 시계열 데이터를 지연 없이 수집하고, 실시간 스트림 분석 및 이상 탐지를 거쳐 영구 저장소와 운영 인터페이스로 전달하는 고신뢰성 파이프라인이 필수적이다¹. 센서 네트워크에서 발생한 계측 신호가 MQTT 프로토콜을 통해 수집되고, IoT 미들웨어 및 Apache Kafka 분산 이벤트 스트리밍 백본을 거쳐 분산 스트림 처리 엔진, 시계열 데이터베이스(TSDB), 그리고 최종 HMI/SCADA 대시보드로 전달되는 구조는 엔터프라이즈 IIoT 참조 모델(Reference Architecture)의 표준적 접근법에 완벽히 부합한다³.

이러한 다계층 파이프라인 구성의 타당성은 계층 간 관심사 분리(Separation of Concerns)와 장애 격리(Fault Isolation)라는 분산 컴퓨팅의 핵심 원칙에 기반한다⁵. 현장 에지 레벨의 경량 네트워크 연결 관리 및 이종 프로토콜 표준화는 전용 수집 계층이 담당하고, 전사 단위의 데이터 내결함성 보장과 고가용성 버퍼링은 Kafka가 전담함으로써 물리적 계측 계층의 불안정성이 상위 분석 및 스토리지 시스템으로 전파되는 현상을 원천적으로 차단한다⁷.

파이프라인 계층	권장 핵심 기술 스택	주요 수행 역할 및 아키텍처적 책임	통신 방식 및 프로토콜
현장 에지 계층	Edge Gateway, 센서 노드	물리량 계측, 산업용 필드버스 신호 변환, 로컬 Store-and-Forward 버퍼링	Modbus, OPC-UA, MQTT ³
IoT 수집 미들웨어	EMQX, EdgeX Foundry, ThingsBoard	디바이스 인증, 프로토콜 정규화, 토픽 체계 추상화, 양방향 제어 명령 라우팅	MQTT v5.0, CoAP, REST ¹⁰
엔터프라이즈 백본	Apache Kafka (KRaft 클러스터)	고신뢰성 대규모 이벤트 로그 버퍼링,	Kafka 바이너리 TCP 프로토콜 ¹³

		파티셔닝 기반 선형 확장, 다중 컨슈머 격리	
실시간 스트림 처리	Apache Flink, ksqlDB	슬라이딩 윈도우 기반 상태 연산, 결측치 실시간 보간, CEP 패턴 매칭, 인라인 ML 추론	Statefun, In-Memory Stream API ¹⁵
공정 시계열 저장소	InfluxDB, TimescaleDB	고빈도 센서 텔레메트리 영구 적재, 태그 기반 고속 인덱싱, 롤업 및 다운샘플링	Influx Line Protocol, SQL ¹⁸
인프라 모니터링	Prometheus, Alertmanager	브로커 랙(Lag), Flink 체크포인트, 호스트 리소스 등 파이프라인 건전성 감시	HTTP Pull (Scrape), PromQL ²⁰
SCADA / 시각화	FUXA, Grafana	P&ID 공정 도면 동적 관제, 양방향 레지스터 제어 발동, 종합 시계열 분석 트렌드 표출	WebSocket, Modbus/TCP, Web API ²²

IoT 미들웨어의 본질적 역할과 메시징 백본 직접 연계의 경계 분석

센서 네트워크에서 송출되는 MQTT 메시지를 엔터프라이즈 파이프라인으로 유입시킬 때, IoT 미들웨어를 경유하는 방식과 Apache Kafka로 직접 파이프라인을 연결하는 방식 사이에는 뚜렷한 기능적·운영적 차이가 존재한다²⁴.

IoT 미들웨어는 단순한 패킷 포워더가 아니며, 물리적 산업 필드 계층과 IT 엔터프라이즈 시스템 간의 구조적 불일치를 해소하는 추상화 계층으로 작동한다³. 산업 현장은 센서 제조사 및 설비 연식에 따라 Modbus-RTU, Modbus-TCP, OPC-UA, BACnet, CAN 등 이종 프로토콜이 난립해 있다³. IoT 미들웨어(예: EdgeX Foundry의 Device Service)는 이러한 산업용 필드버스 신호를 단일화된 JSON 또는 Protocol Buffers 형태의 시맨틱 데이터 모델로 정규화하는 작업을 수행한다²⁶. 또한 디바이스의 상호 인증(mTLS, 토큰 기반 검증), 등록 라이프사이클 관리, 센서의 캘리브레이션 계수 및 물리적 설치 좌표와 같은 디지털 트윈 메타데이터를 유지 관리한다¹⁰.

결정적으로, 센서나 PLC 레지스터에 특정 제어 값을 주입하는 사우스바운드(Southbound) 양방향 제어 경로를 수립하고 유지하는 주체는 Kafka가 아닌 IoT 미들웨어다²⁷. 반면, 현장 센서들이 이미 균일한 스펙을 갖추고 표준화된 MQTT 토픽으로 데이터를 자율 발행하고 있으며, 시스템의 요구사항이 단방향 데이터 수집 및 로깅에 국한되어 있다면 IoT 미들웨어를 거치지 않고 MQTT 브로커에서 Kafka로 직결하는 아키텍처가 유리하다¹². EMQX와 같은 고성능 분산 MQTT 브로커는 내부 규칙 엔진(Rule Engine)과 내장 Kafka 프로듀서 커넥터를 제공하여, 중간 변환 오버헤드 없이 수십만 TPS의 데이터를 Kafka 토픽으로 즉각 분배할 수 있다¹².

그러나 설비 제어 명령을 하향 전파해야 하거나, 장비의 인가 권한을 동적으로 제어해야 하는 SCADA 본연의 기능을 수행해야 한다면 IoT 미들웨어를 진입점에 배치하고, 정제된 이벤트 스트림만을 Kafka로 토출시키는 다계층 구조를 유지하는 것이 아키텍처적 결함을 방지하는 유일한 방안이다⁴.

데이터 무결성 보장을 위한 신뢰성 수집 체계 및 결측치 보관 전략

무손실 데이터 수집을 위한 전송 계층 신뢰성 메커니즘

무선 센서 네트워크나 광역 공장망은 통신 간섭, 전원 노이즈, 스위치 리부팅 등으로 인한 간헐적 네트워크 단절에 취약하다³⁰. 센서 패킷의 누락 없는 수집을 위해서는 프로토콜과 에지 노드 양단에서 다중 안전장치가 가동되어야 한다.

MQTT 프로토콜 수준에서는 신뢰성 보장을 위해 QoS(Quality of Service) 레벨 관리가 필수적이다. 산업 현장에서는 네트워크 오버헤드가 과도한 QoS 2(정확히 1회)보다는 고유 시퀀스 번호와 타임스탬프를 페이로드에 포함한 QoS 1(최소 1회 전달) 모드를 채택하고, 중복 수신 문제는 스트림 처리 계층의 멱등성(Idempotency) 로직으로 해소하는 것이 표준적이다⁵. 이와 함께 브로커와 센서 클라이언트 간의 세션 영속성(Persistent Session)을 활성화하여, 일시적인 물리 계층 단절 시에도 브로커가 해당 클라이언트를 위한 미전송 메시지를 인메모리 또는 디스크 큐에 보존하도록 설정해야 한다¹¹.

보다 근본적인 유실 방지 대책은 에지 게이트웨이 레벨의 로컬 역단절(Store-and-Forward) 메커니즘 구축이다⁹. 광역 네트워크가 완전히 차단되면 브로커의 세션 버퍼는 곧 한계에 도달하므로, 현장 게이트웨이는 인입되는 센서 프레임을 로컬 SQLite, RocksDB, 또는 플래시 메모리 파일 시스템에 트랜잭션 단위로 선기록(Store)해야 한다⁹. 중앙 서버와의 연결이 재수립되면 에지 에이전트는 로컬 큐에 보관된 이력 데이터를 타임스탬프 순서대로 순차 재전송(Forward)하여 통신 단절 기간의 데이터 공백을 무손실 복구한다³².

센서 결측치 발생 특성과 머신러닝 연동 시의 전처리 필수성

센서 데이터의 결측치는 패킷 드롭뿐만 아니라 센서 하드웨어 결함, 급격한 전압 강하, 센서 헤드 캘리브레이션 지연 등 복합적인 요인에 의해 유발된다³⁰.

머신러닝이나 딥러닝 기반 분석을 수행하지 않고 단순 모니터링 및 시각화 대시보드 운영만을 목표로 하는 경우, 데이터 파이프라인 내부에서 결측치를 인위적으로 보관할 당위성은 낮다.

InfluxDB나 TimescaleDB 같은 시계열 데이터베이스 엔진은 쿼리 실행 시점에 직전 값 유지(LOCF)나 선형 보간 함수를 자체적으로 수행하여 공백을 메우는 기능을 지원한다¹⁹. 원천 데이터 레벨에서 인위적으로 계산된 가공치를 덮어쓰게 되면 실제 물리 계측치와 추정치 간의 경계가 모호해져 향후 사고 발생 시 감사 추적(Audit Trail)의 법적·기술적 신뢰성을 훼손할 수

있기 때문이다.

반면 **Autoencoder, LSTM, Isolation Forest** 등 실시간 다변량 센서 데이터를 기반으로 작동하는 머신러닝 모델을 파이프라인에 통합할 경우, 결측치 보간은 반드시 거쳐야 하는 필수 전처리 공정이 된다¹⁵. 딥러닝 모델의 신경망 레이어는 입력 데이터가 고정된 차원의 완전한 텐서(Tensor) 행렬 형태로 연속 유입될 것을 전제한다. 만약 입력 벡터에 결측을 의미하는 NaN 이나 공백 타임스탬프가 유입되면 부동소수점 연산 오류가 발생하거나, 비정상 패턴이 아님에도 불구하고 모델의 복원 손실률(Reconstruction Loss)이 폭증하여 시스템 전반에 치명적인 오탐(False Positive)을 유발하기 때문이다³⁶.

결측치 보간 기법	수식 및 알고리즘 원리	장점	단점 및 제약사항	최적 배치 위치
직전 관측치 대체 (LOCF)	$\hat{x}_t = (x_{t-k})$ 는 결측 직전 최신 유효 관측값)	연산 복잡도 $O(1)$, 메모리 소모 극소, 실시간 스트림에 즉시 적용 가능	시계열의 동적 변화 추세가 완전히 단절되며 지연 오차 발생	Edge Gateway, Stream Engine ³⁶
국소 선형 보간 (Linear)	$\hat{x}_t = x_A + \frac{t-t_A}{t_B-t_A}(x_B-x_A)$ (시간차 비례 보간)	온도, 레벨 등 연속적인 물리량 변화에 대해 높은 물리적 타당성 제공	미래 시점 관측치(t_B)가 도착할 때까지 스트림 버퍼링 지연 발생	Stream Engine (Sliding Window) ³⁷
다항 곡선 적합 회귀 (Spline/Polynomial)	국소 윈도우 내 n 차 다항식 곡선 회귀 $f(t) =$	기계 진동이나 회전체 속도 등 주기적·곡선형 물리량 패턴의 정밀 보존	연산 오버헤드 증가, 윈도우 경계면 발산 현상(Runge 현상) 위험	Central Stream Engine (Apache Flink) ³⁸
다변량 공간 회귀 (Spatial Regression)	$\hat{x}_{i,t} =$ (상관 센서 회귀)	특정 센서의 장시간 데이터 단절 시에도 주변 상관 센서로 정확한 복원	센서 간 상관관계 행렬 유지 비용, 상관 센서 동시 단절 시 모델 붕괴	Central Stream Engine (Stateful) ³⁹

결측치 보간을 실행하기 위한 가장 적합한 위치는 **Kafka**와 시계열 데이터베이스 사이에

배치되는 분산 스트림 처리 계층(Apache Flink)이다³⁶. 에지 게이트웨이는 수백 개 센서의 다변량 상관 회귀나 슬라이딩 윈도우 상태를 유지하기에 메모리와 연산 능력이 제한적이며, TSDB에 일단 적재된 이후에 보간을 수행하는 방식은 실시간 이상 탐지가 요구하는 수십 밀리초 단위의 종단 간 지연시간(End-to-End Latency) 요건을 충족할 수 없다.

복합 이벤트 처리(CEP) 엔진 분석: Esper/EQL vs ksqldb vs Apache Flink

센서 데이터 스트림 내에서 단순 수치 비교를 넘어, 특정 이벤트의 발생 순서나 시간적 상관관계를 규명하기 위해 복합 이벤트 처리(Complex Event Processing, CEP) 엔진의 도입이 검토된다¹⁷. 과거 임베디드 아키텍처에서 널리 활용된 Esper(EQL 기반)와 현대 분산 스트리밍 생태계의 기술적 상관관계를 면밀히 평가해야 한다.

Esper는 이벤트 구동 아키텍처(EDA)를 구축하기 위해 개발된 고성능 인메모리 CEP 엔진으로, SQL과 유사한 이벤트 쿼리 언어(EQL)를 사용하여 시간 윈도우, 패턴 매칭(예: "A 이벤트 발생 후 3분 이내에 B가 발생하지 않는 상황"), 인과관계 필터링을 직관적으로 선언할 수 있다¹⁷. 그러나 Esper는 태생적으로 단일 프로세스 JVM 내에 임베디드 라이브러리 형태로 내장되어 구동되도록 설계되었다¹⁷. 이로 인해 초당 수십만 건의 대규모 트래픽이 집중되거나 고가용성 클러스터링이 요구되는 엔터프라이즈 환경에서는 노드 장애 시 상태 복구와 수평적 확장(Scale-out)에 치명적인 한계를 드러낸다.

현재의 대규모 분산 파이프라인에서는 Esper를 독립 컴포넌트로 유지하기보다는, Kafka 생태계와 네이티브하게 통합되는 ksqldb나 Apache Flink를 사용하는 것이 표준적이다⁵.

ksqldb는 Kafka 상단에서 직관적인 스트림 조인, 필터링, 윈도우 집계를 순수 SQL 문법으로 처리할 수 있어 시스템 도입 장벽이 매우 낮다¹⁷. 반면 다중 이벤트 간의 정합적 순서 패턴을 감지하는 순수 CEP 작업이나 커스텀 텐서 조작에는 구조적 한계가 존재한다.

Apache Flink는 Flink CEP 라이브러리를 통해 Esper의 EQL 수준에 필적하는 고도의 비결정적 유한 오토마타(NFA) 기반 패턴 매칭 기능을 분산 환경에서 제공한다⁵. 더욱이 Flink는 RocksDB 기반의 분산 상태 저장소와 Chandy-Lamport 알고리즘 기반의 비동기 체크포인팅을 지원하여 노드 장애 시에도 '정확히 한 번(Exactly-once)' 상태 복구를 완벽히 보장한다⁵.

따라서 현대적인 아키텍처 관점에서 Esper를 별도로 배치하여 파이프라인의 복잡도를 가중시킬 필요가 없으며, Apache Flink(또는 상대적으로 단순한 SQL 윈도우 연산일 경우 ksqldb)를 Kafka에 직접 컨슈머로 결합하여 결측치 보간, 스트림 정제, 복합 이벤트 탐지를 단일 분산 계층에서 통합 수행하는 구성이 가장 합리적이다¹⁶.

시계열 저장소 및 메트릭 모니터링 체계: InfluxDB와

Prometheus의 역할 분리

파이프라인 설계에서 가장 심각한 병목과 장애를 초래하는 원인 중 하나는 Prometheus와 InfluxDB의 기능적 중복 및 역할 혼선이다. 두 시스템은 모두 시간에 따라 변화하는 메트릭을 다루지만, 아키텍처 설계 철학과 데이터 모델, 쓰기/읽기 패턴이 근본적으로 다르다¹⁸.

Prometheus는 분산 IT 인프라, 컨테이너, 마이크로서비스의 가용성 감시 및 알람을 위해 특화된 시스템이다²⁰. 대상 시스템의 엔드포인트(/metrics)를 정기적으로 폴링하는 주기적 HTTP Pull(Scraping) 방식을 강제하며, 저장 가능한 데이터 형태 역시 고정된 단일 Float64 수치로 극히 제한된다²⁰. 무엇보다 Prometheus의 로컬 시계열 데이터베이스는 시계열의 식별 태그 조합 수가

기하급수적으로 증가하는 고 카디널리티(High Cardinality) 환경에 노출될 경우 인덱스 크기가 RAM 용량을 초과하여 프로세스 충돌(OOM Crash)을 일으킨다²⁰.

반면 산업 현장의 센서 데이터는 공정 상황에 따라 밀리초 단위로 자율 푸시(Push)되는 불규칙한 이벤트 스트림이며, 부동소수점 외에도 장비 에러 코드, 운전 모드 문자열, 다차원 JSON 페이로드 등 다양한 데이터 타입을 포함한다²⁰. 또한 센서 시리얼 넘버, 공정 라인 번호, 설비 태그 등 수백만 개 이상의 조합을 갖는 극단적인 고 카디널리티 구조를 형성한다¹⁹.

분석 지표	Prometheus	InfluxDB
시스템 본질적 목적	IT 인프라, 컨테이너, 서비스 계층 모니터링 및 실시간 알림 ²⁰	대규모 산업 센서 텔레메트리 적재, 고성능 프로세스 히스토리안 ¹⁸
데이터 수집 아키텍처	Pull 메커니즘 (정해진 주기의 HTTP 타겟 스크랩) ⁴⁴	Push 메커니즘 (Line Protocol 기반 비동기 벌크 쓰기) ¹⁹
지원 데이터 타입	단일 Float64 (문자열, 복합 구조체, 바이너리 저장 불가) ²⁰	Float, Integer, String, Boolean, 다차원 필드셋 ²⁰
카디널리티 수용 한계	고 카디널리티에 극도로 취약 (메모리 고갈 및 인덱스 붕괴 위험) ²⁰	시계열 전용 스토리지 엔진 기반 고 카디널리티 최적화 ¹⁹
타임스탬프 해상도	밀리초(Millisecond) 단위, 스크랩 주기에 종속적 ²⁰	나노초(Nanosecond) 단위 정밀 타임스탬프 지원 ¹⁹
데이터 보존 주기	단기~중기 (기본 수일~수개월, 인프라 메트릭용) ¹⁸	장기 영구 보관 (수년 이상의 연속 공정 이력 보존 및 다운샘플링) ¹⁸

이러한 특성으로 인해 Prometheus를 센서 데이터 수집의 영구 저장소로 채택하는 것은 불가능하며, 두 컴포넌트는 철저하게 이원화되어 운영되어야 한다¹⁸.

InfluxDB(또는 대규모 분석 워크로드에 최적화된 TimescaleDB)는 Flink나 Kafka Connect를 통해 실시간 유입되는 공정 센서 데이터를 초당 수십만 건 이상 안전하게 적재하는 프로세스 히스토리안(Process Historian) 역할을 담당해야 한다¹⁸.

반면 Prometheus는 순수한 파이프라인 인프라 가용성 감시 계층으로 완전히 격리되어야 한다²⁰. 즉, IoT 게이트웨이의 CPU 부하, EMQX 브로커의 세션 유지 수, Kafka 클러스터의 토픽별 컨슈머 락(Consumer Lag), Flink 잡매니저의 체크포인트 소요 시간, InfluxDB의 디스크 I/O 등을 모니터링하고, 파이프라인 자체에 장애가 발생했을 때 Alertmanager를 통해 운영진에게 즉각적인 경보를 발송하는 역할에 국한되어야 한다²¹.

다층 이상 징후 탐지 메커니즘과 스트림 머신러닝 연동 아키텍처

이상 징후 탐지(Anomaly Detection)는 시스템 리소스와 감지 대상 이상의 복잡도에 따라 정적 규칙 기반 탐지와 동적 머신러닝 기반 탐지의 다계층 구조로 설계되어야 한다.

1계층: 정적 규칙 및 통계 기반 탐지

명확한 물리적 한계치나 급격한 이상 상태를 감지할 때 복잡한 신경망 모델을 사용하는 것은 불필요한 연산 낭비를 초래한다. 1계층에서는 결정론적(Deterministic) 규칙과 국소 통계치를 활용하여 수 밀리초 이내의 초저지연 알람을 발생시킨다³⁹.

- 엔지니어링 임계치(USL/LSL) 감지: 압력 챔버나 반응기의 허용 압력 한계 등 사전에 정의된 상한 규격치(USL) 또는 하한 규격치(LSL)를 센서 계측값이 벗어나는 순간 즉각 알람을 트리거한다.
- 동적 롤링 Z-Score 감지: 센서별 슬라이딩 윈도우 내에서 실시간 평균(μ_w)과 표준편차(σ_w)를 계산하고, 현재 계측치 x_t 의 표준편차 이탈 정도를 산출한다³⁹.

$$Z = \frac{x_t - \mu_w}{\sigma_w}$$

산출된 $|Z|$ 값이 임계점(예: 3.0)을 초과할 경우 공정 환경의 급격한 이상 변동으로 판정한다³⁹.

- 복합 상태 규칙(CEP Pattern): "모터 전류가 정격의 120%를 초과한 후 10초 이내에 베어링 진동 센서의 진폭이 기준치를 상회하는 경우"와 같이 여러 센서 간의 시간적 종속 관계를 ksqldb나 Flink CEP를 통해 인메모리로 실시간 추적한다¹⁷.

2계층: 머신러닝/딥러닝 기반 이상 탐지 모델

단일 센서의 임계치 초과로는 식별할 수 없는 센서 간의 비선형적 상호작용 왜곡이나 설비의 미세한 마모 및 열화(Degradation)는 비지도 학습(Unsupervised Learning) 기반의 딥러닝 모델을 통해 포착해야 한다¹⁵.

대표적인 알고리즘으로 Autoencoder 및 시계열 특화 LSTM Autoencoder가 널리 채택된다³⁵. 이 모델은 정상 운전 상태에서 수집된 다변량 센서 데이터를 입력받아 인코더(Encoder)를 통해 저차원 잠재 벡터로 압축한 후, 디코더(Decoder)를 거쳐 원래의 정상 신호로 복원하도록 사전 학습된다³⁵. 공정에 비정상 징후가 발생하면 학습된 정상 패턴과의 괴리로 인해 모델이 입력

신호를 제대로 복원하지 못하며, 이로 인해 원본 신호 x_t 와 복원 신호 $\hat{x}_t$ 간의 재구성 오차(Reconstruction Error)가 급격히 증가한다¹⁵.

$$e_t = \|x_t - \hat{x}_t\|_2^2 = \sum_{i=1}^M (x_{t,i} - \hat{x}_{t,i})^2$$

산출된 재구성 오차 e_t 가 사전에 최적화된 이상 점수 임계치 τ 를 초과할 때, 모델은 설비에 잠재적 이상이 발생했음을 확정하고 이상 기여도가 높은 특정 센서 차원을 역추적한다.

스트리밍 파이프라인 내 머신러닝 모델 서빙 표준 구현 방식

머신러닝 알고리즘을 실시간 스트리밍 데이터 파이프라인에 탑재할 때, 모델의 오프라인 학습 주기와 온라인 추론 주기는 철저히 분리되어야 한다⁴⁸.

오프라인 학습 단계에서는 InfluxDB 또는 장기 데이터 레이크에 누적된 정상 운전 이력 데이터를 Python(PyTorch, TensorFlow, Scikit-learn) 기반 분산 학습 클러스터로 로드하여 Autoencoder 모델을 훈련시킨다⁴⁹. 학습이 완료된 딥러닝 신경망은 오픈 표준 바이너리 포맷인 ONNX(Open Neural Network Exchange) 파일로 익스포트되어 중앙 모델 레지스트리에 배포된다⁵⁰.

실시간 온라인 추론 단계에서는 외부 REST/gRPC API 서버를 호출하는 방식(RPC Callout)을 지양해야 한다. 네트워크 홉으로 인한 지연시간 증가와 API 서버 병목 현상을 방지하기 위해, Apache Flink 태스크 매니저 프로세스 내부에 C++ 기반의 고성능 ONNX Runtime 엔진을 임베디드(In-Stream Embedded Serving) 형태로 직접 로드한다⁴⁸.

Flink는 Kafka로부터 센서 스트림을 실시간 소비하면서 결측치를 보간하고 텐서 슬라이딩 윈도우를 형성한 직후, 로컬 인메모리 상에서 ONNX 모델을 직접 호출하여 수 밀리초 이내에 이상 점수를 판출한다⁴⁸. 이상으로 판정된 이벤트는 Kafka의 별도 알림 토픽(sensor.alerts)으로 재발행되어 운영자 대시보드로 즉시 전파된다⁵⁴.

SCADA 시각화 레이어의 실체적 분석: Grafana의 기능적 한계와 오픈소스 웹 SCADA

Grafana의 명확한 한계와 오적용 분석

Grafana는 시계열 데이터베이스를 조회하여 다차원 메트릭을 동적으로 표출하고 트렌드를 분석하는 시각화 도구로서는 최고의 성능을 발휘한다²². 그러나 전통적인 산업 플랜트 제어 관점에서 Grafana만을 단독 배치하고 이를 "완전한 SCADA 시스템"으로 정의하는 것은 중대한 아키텍처적 결함을 내포한다²².

SCADA의 본질은 감시(Supervisory)뿐만 아니라 제어(Control)에 있다. Grafana는 기본적으로 시계열 저장소의 데이터를 읽어오는 단방향 읽기 전용(Read-only) 대시보드 구조를 지향한다²². 버튼 클릭을 통해 특정 PLC의 코일 레지스터를 조작하거나 밸브 개도율을 변경하는 양방향 명령 하달(Write-back) 기능이 네이티브로 제공되지 않는다. 외부 API를 호출하는 비공식 플러그인을 적용할 수 있으나, 산업용 제어에서 필수적인 물리적 안전 인터록(Interlock), 쓰기 확인(Echo Check), 명령 실행 감사 로깅 등의 결정론적 안전 메커니즘을 보장할 수 없다.

또한 Grafana의 패널 체계는 시간 축 기반의 꺾은선 그래프, 바 차트, 게이지 표현에 고착되어 있다. 실제 플랜트 운영자가 필요로 하는 공정 계장 도면(P&ID), 배관 내 유체 흐름 애니메이션, 펌프 및 밸브의 물리적 레이아웃 기반 동적 상태 점멸(Mimic Diagram)을 엔지니어링 표준에 맞추어 시각화하는 데 심각한 제약이 따른다²².

진정한 오픈소스 웹 SCADA 아키텍처와의 결함

이러한 한계를 극복하고 완전한 오픈소스 감시 제어 시스템을 완성하기 위해서는 Grafana와 함께 전문 웹 SCADA 엔진을 병행 배치하는 하이브리드 구성이 요구된다⁵⁷.

FUXA는 순수 웹 브라우저 환경에서 동작하는 차세대 오픈소스 SCADA/HMI 플랫폼이다²². 엔지니어는 웹 GUI 상에서 드래그 앤 드롭 방식으로 산업용 SVG 심볼과 P&ID 도면을 배치하고,

모터 회전이나 배관 흐름 애니메이션을 직관적으로 바인딩할 수 있다²². 더욱이 FUXA는 Modbus, OPC-UA, MQTT 프로토콜 클라이언트를 자체 엔진에 탑재하고 있어, 공정 감시 도면에서 운영자가 버튼을 클릭하면 현장 게이트웨이나 PLC의 특정 주소 번지로 제어 명령을 직접 하향 전송하는 양방향 SCADA 루프를 완벽히 구현한다²².

ThingsBoard 역시 자체적인 웹 HMI 대시보드 빌더를 내장하고 있으며, 슬라이더, 토글스위치, 제어 노브 등 양방향 RPC(Remote Procedure Call) 위젯을 통해 센서 및 구동기(Actuator)로 즉각적인 제어 페이로드를 전달하는 기능을 안정적으로 제공한다²⁷.

결과적으로, 현장 설비의 실시간 상태 확인 및 양방향 기기 제어는 FUXA 또는 ThingsBoard 대시보드에 일임하고, 장기 시계열 히스토리안 트렌드 분석, 설비 종합 효율(OEE) 통계, 그리고 전체 데이터 파이프라인의 인프라 건전성 모니터링은 Grafana가 담당하도록 이원화하는 것이 엔지니어링 관점에서 가장 이상적인 구성이다²².

기 구축된 오픈소스 IoT 플랫폼 평가 및 사용자 스택 비교

센서 수집부터 엔터프라이즈 백본, 스트림 처리, 분석에 이르는 전 주기를 밑바닥부터(From Scratch) 독자 구현할 수도 있으나, 검증된 오픈소스 IIoT 플랫폼들의 기능적 특성을 이해하고 요구사항에 맞추어 선별적으로 결합하는 것이 개발 및 유지보수 비용을 획기적으로 절감시킨다²⁴.

ThingsBoard는 장치 관리, MQTT/CoAP/HTTP 전송 엔드포인트, 드래그 앤 드롭 룰 엔진(Rule Engine), 대시보드를 단일 솔루션으로 제공하는 종합 IoT 플랫폼이다¹⁰. 별도의 외부 분석 엔진을 구성하지 않고도 수집된 데이터를 즉시 필터링하거나 경보를 발생시킬 수 있으며, Rule Engine 내에 내장된 Apache Kafka 노드를 통해 특정 조건의 텔레메트리를 엔터프라이즈 Kafka 토픽으로 쉽게 바이패스할 수 있다¹³. 그러나 플랫폼의 모놀리식/마이크로서비스 구조 자체가 무겁고, 초당 수십만 건 이상의 초고빈도 스트림이 유입될 경우 플랫폼 내부 메시지 큐의 오버헤드가 발생할 수 있다⁶.

EdgeX Foundry는 리눅스 재단(LF Edge) 산하의 마이크로서비스 기반 에지 컴퓨팅 플랫폼이다⁴. 상위 시각화보다는 현장 에지 컴퓨터 내에서의 사우스바운드 프로토콜 통합에 특화되어 있다²⁶. Modbus, OPC-UA 등 산업용 장비의 데이터를 완벽하게 정규화하여 로컬 코어 서비스로 수집한 후, eKuiper와 같은 초경량 룰 엔진을 거쳐 클라우드나 엔터프라이즈의 Apache Kafka, EMQX로 데이터를 안전하게 릴레이하는 '에지 수집 게이트웨이'로서 독보적인 완성도를 갖는다¹².

Apache StreamPipes는 산업용 IoT 데이터 스트림의 분석을 노코드(No-code) 환경으로 구축하기 위해 탄생한 오픈소스 플랫폼이다⁶². 센서 데이터 어댑터, 윈도우 변환 필터, 이상 탐지 알고리즘 등을 웹 파이프라인 에디터 상에서 그래픽 블록 조립 형태로 구성할 수 있어 현장 도메인 엔지니어의 접근성이 매우 높다⁶². 그러나 고성능 분산 분기 연산이나 복잡한 딥러닝 텐서 처리에는 제약이 따른다.

비교 항목	ThingsBoard (Community)	EdgeX Foundry	Apache StreamPipes	사용자 제안 스택 (Flink-Kafka-F UXA)

핵심 포지셔닝	중앙 집중형 통합 디바이스 관리 ⁶	에지 레벨 산업 프로토콜 정규화 ²⁹	노코드 산업용 스트림 데이터 분석 ⁶²	엔터프라이즈 급 대규모 분산 파이프라인
데이터 수집 백본	내장 큐 또는 외부 Kafka 플러그인 ¹⁴	내부 eKuiper/ZeroMQ 또는 MQTT ²⁶	내장 Kafka/Pulsar 엔진 기반 ⁶⁴	순수 Kafka KRaft 분산 클러스터 ¹³
산업 프로토콜 수용성	MQTT, CoAP, HTTP, LwM2M 중심 ¹⁰	Modbus, OPC-UA, BACnet 등 극대화 ³	전용 커넥터를 통한 PLC/MQTT 수집 ⁶⁴	IoT 미들웨어(Edge X/EMQX) 어댑터 연동
스트림 분석 및 ML	노드 기반 비주얼 룰 엔진 ¹⁰	eKuiper 연계 경량 SQL 스트림 처리 ¹²	사전 정의된 스트림 분석 블록 ⁶²	Flink 기반 분산 상태 연산 및 ONNX 실시간 서빙 ⁴⁸
SCADA 제어 적합성	양방향 RPC 제어 위젯 완비 ²⁷	REST 기반 Command 마이크로서비 스 ²⁶	분석 위주 (현장 제어 기능 부재)	FUXA 결합을 통한 정밀 P&ID 양방향 제어 ²²
분산 확장성 및 성능	중간~높음 (컨테이너 클러스터 배포) ⁶	단일 에지 게이트웨이 리소스에 종속	중간 (독립 클러스터링 지원)	극대화 (Kafka 파티션 및 Flink 슬롯 선형 확장) ⁵

종합적으로 검토할 때, 사용자가 구상한 기술 조합(MQTT -> 수집 미들웨어 -> Kafka -> Flink/ksqlDB -> InfluxDB -> Grafana/FUXA)은 특정 상용 프레임워크에 종속되지 않으면서도, 시스템 처리 용량을 무제한으로 수평 확장하고 머신러닝 추론을 초저지연으로 결합할 수 있는 엔터프라이즈급 최상위 아키텍처에 해당한다³.

엔터프라이즈급 최종 권장 아키텍처 및 단계별 구현 로드맵

사용자의 아이디어를 실제 운영 가능한 고신뢰성 시스템으로 구체화하기 위한 엔드투엔드 파이프라인의 물리적 데이터 흐름과 계층별 구현 세부 명세는 다음과 같다.

현장 에지 계층(Tier 1)에서는 공장 라인의 PLC 및 센서가 Modbus나 OPC-UA 프로토콜을 통해 에지 게이트웨이로 물리적 신호를 전송한다³. 게이트웨이에 설치된 EdgeX Foundry는 장비 드라이버를 통해 수집된 바이너리 데이터를 단일 JSON 스키마로 파싱한다²⁶. 만약 외부 네트워크가 단절되면 로컬 SQLite 기반의 Store-and-Forward 모듈이 활성화되어 데이터를 디스크에 누적하고, 통신 복구 시 MQTT QoS 1 및 세션 유지를 적용하여 상위 브로커로 무손실 재전송한다⁹.

엔터프라이즈 수집 및 백본 계층(Tier 2)에서는 클러스터링된 EMQX 브로커가 수천 대의 에지 게이트웨이로부터 암호화된(TLS) MQTT 스트림을 중단 수신한다¹². EMQX의 데이터 통합 브리지를 통해 수신된 센서 텔레메트리는 지체 없이 Apache Kafka 클러스터의 `sensor.telemetry.raw` 토픽으로 적재된다¹². 이로써 전사 차원의 완충 큐가 확보되고 하위 시스템과의 물리적 의존성이 완전히 분리된다⁸.

실시간 분산 스트림 처리 및 머신러닝 계층(Tier 3)에서는 분산 Apache Flink 클러스터가 Kafka의 원천 데이터를 고속 소비한다⁵. Flink는 RocksDB 기반의 상태 윈도우를 열어 센서 타임스탬프를 검사하며, 패킷 유실로 인한 결측 구간을 슬라이딩 윈도우 선형 보간 또는 LOCF 알고리즘으로 실시간 채워 넣는다⁵. 보간이 완료된 완전한 데이터 벡터는 Flink 태스크 매니저에 내장된 C++ ONNX Runtime으로 전달된다⁴⁸. 사전 학습된 Autoencoder 딥러닝 신경망은 1밀리초 미만의

지연시간으로 재구성 오차 e_t 를 계산하여 이상 점수를 도출한다⁴⁸. 정제된 센서 스트림은 `sensor.telemetry.clean` 토픽으로, 임계치를 초과한 비정상 이벤트는 `sensor.alerts` 토픽으로 각각 분기되어 Kafka에 재발행된다⁵⁴.

저장 및 SCADA 시각화 계층(Tier 4)에서는 두 갈래의 가시화 및 제어 파이프라인이 동시에 작동한다. 정제된 공정 텔레메트리는 Kafka Connect 또는 Telegraf 컨슈머를 통해 InfluxDB(또는 TimescaleDB)로 배치 단위 고속 적재되어 장기 공정 히스토리안을 구축한다¹⁸.

현장 공정 운전원은 FUXA 웹 SCADA를 통해 P&ID 도면 상에서 밸브, 펌프의 실시간 상태를 동적 애니메이션으로 확인하고, 필요 시 Modbus/TCP 또는 OPC-UA를 통해 PLC에 직접 양방향 제어 신호를 인가한다²². 공장 관리자와 데이터 사이언티스트는 Grafana를 통해 InfluxDB에 축적된 장기 트렌드와 Flink가 도출한 머신러닝 이상 징후 이력을 교차 분석하며, 동시에 Prometheus를 통해 Kafka의 컨슈머 랙 및 Flink 클러스터의 인프라 건전성을 통합 감시한다²⁰.

종합 결론

검토 결과, 사용자가 제안한 데이터 파이프라인의 기본 구상은 최신 분산 엔터프라이즈 IIoT 아키텍처의 핵심 원칙을 정확히 관통하고 있다³. 시스템의 안정적인 상용화를 위한 핵심 의사결정 권고사항은 다음과 같이 집약된다.

1. **IoT** 미들웨어의 존치: 센서 네트워크와 Kafka 사이에 EdgeX Foundry나 고성능 브로커(EMQX)를 반드시 유지하여 프로토콜 정규화와 에지 Store-and-Forward 유실 방지 체계를 수립해야 한다⁴.
2. **Esper**의 현대적 대체: 레거시 단일 JVM 엔진인 Esper/EQL 대신, Kafka와 완벽히 결합하여 수평 확장과 무손실 상태 복구를 보장하는 Apache Flink(Flink CEP)를 스트림 처리의 핵심 코어로 채택해야 한다⁵.
3. 선택적 결측치 보간: 단순 시각화용 데이터는 무결성을 위해 결측 상태를 그대로 보존하되, 실시간 머신러닝 모델이 개입하는 분석 경로에 한하여 Flink 스트림 내부에서 선형 또는 회귀 보간을 선행시켜야 한다¹⁹.
4. 인라인 머신러닝 추론 서빙: 딥러닝 이상 탐지 모델(Autoencoder)은 Python 환경에서 사전 학습 후 ONNX로 직렬화하여, Flink 내부에서 임베디드 라이브러리(ONNX Runtime) 형태로 초저지연 구동해야 한다⁴⁸.
5. 저장소 및 **SCADA** 인터페이스의 이원화: 시계열 센서 데이터 히스토리안은 InfluxDB에 위임하고 Prometheus는 순수 파이프라인 인프라 모니터링으로 격리해야 하며, 시각화 역시 장기 트렌드 분석의 Grafana와 양방향 공정 제어 및 P&ID 계장을 지원하는 FUXA 웹

SCADA로 역할을 엄격히 분담해야 한다¹⁸.

참고 자료

1. (PDF) Implementation and Experimental Application of Industrial IoT, https://www.researchgate.net/publication/387210322_Implementation_and_Experimental_Application_of_Industrial_IoT_Architecture_Using_Automation_and_IoT_HardwareSoftware
2. Scalable Real-time Anomaly Detection for Distributed Sensor Systems, <https://oaji.net/pdfs.html?n=2026/3603-1781854915.pdf>
3. IoT Development Services & IoT Security Solutions | ISpectra, <https://ispectratechnologies.com/services/iot.html>
4. A Five-Layer Open-Source IoT Platform Architecture for Smart City, <https://www.mdpi.com/2079-9292/15/15/3377>
5. A Synergistic Elixir-EDA-MQTT Framework for Advanced Smart, <https://www.mdpi.com/1999-5903/16/3/81>
6. Architecture Overview | Docs | ThingsBoard PE, <https://thingsboard.io/docs/pe/reference/architecture/>
7. 9 Best Open-Source IoT Platforms Compared in 2026 - Promeraki, <https://promeraki.com/blog/open-source-iot-platforms>
8. Microservices Architecture | Docs | ThingsBoard PE, <https://thingsboard.io/docs/pe/reference/architecture/microservices/>
9. Best Adafruit IO Alternatives & Competitors - SourceForge, <https://sourceforge.net/software/product/Adafruit-IO/alternatives>
10. Survey of IoT Platforms and Tools, https://wjarr.com/sites/default/files/fulltext_pdf/WJARR-2022-0064.pdf
11. An overview of IoT architectures, technologies, and existing ... - arXiv, <https://arxiv.org/pdf/2401.15441>
12. Edge Cloud Collaboration | eKuiper Documentation, https://ekuiper.org/docs/en/latest/integrations/edge_cloud/overview.html
13. Kafka Integration | Docs | ThingsBoard PE, <https://thingsboard.io/docs/pe/user-guide/integrations/kafka/>
14. Message Queue | Docs | ThingsBoard PE, <https://thingsboard.io/docs/pe/reference/architecture/queue/>
15. Integrating Real Time Monitoring and Automated Anomaly Detection, <https://journal.apjikom.or.id/index.php/BDAS/article/download/21/24>
16. Real-Time ETL with Flink and Kafka: Trends | by Sanket Nadargi, <https://medium.com/@sanket.nadargi1/real-time-etl-with-flink-and-kafka-trends-ebb3d9dff635>
17. Streaming event correlation and complex event processing using, <https://digikogu.taltech.ee/en/Download/a2b1dc4e-f131-4174-b753-b0ca4582f4f9>
18. (PDF) Digital twins and IIoT: comparison of Prometheus and InfluxDB, https://www.researchgate.net/publication/396361482_Digital_twins_and_IIoT_comparison_of_Prometheus_and_InfluxDB

19. InfluxDB vs TimescaleDB: Purpose-Built or SQL (2026),
<https://www.modern-datatools.com/compare/influxdb-vs-timescaledb>
20. Prometheus, InfluxDB, and TimescaleDB - The Basic Tech Info,
<https://www.thebasictechinfo.com/data-persistence/time-series-databases-explained-prometheus-influxdb-and-timescaledb/>
21. 5 InfluxDB Alternatives - Tiger Data,
<https://www.tigerdata.com/learn/5-influxdb-alternatives-for-your-time-series-data>
22. FUXA: A Web-Based Configuration Tool for SCADA/HMI | ZedIoT,
<https://zediot.com/blog/fuxa-a-web-based-configuration-tool-for-scada-hmi/>
23. Add support for InfluxDB · Issue #32 · frangoteam/FUXA - GitHub,
<https://github.com/frangoteam/FUXA/issues/32>
24. Top 30 Best Open Source IoT Platform Picks in 2026,
<https://techtidesolutions.com/rankings/best-open-source-iot-platform/>
25. TBMQ 2.1: New chapter in MQTT messaging with embedded,
<https://thingsboard.io/blog/tbmq-2-1-new-chapter-in-mqtt-messaging-with-embedded-integrations/>
26. Edge Central Value Add - IOTech Systems,
<https://www.iotechsys.com/wp-content/uploads/2026/01/edge-central-value-add.pdf>
27. Top Free & Open-Source IIoT Platforms for Smart Manufacturing,
<https://mdcplus.fi/blog/top-free-iiot-systems-interesting-of-things-open-source/>
28. "A whole new world of possibilities": interview with Edge architect,
<https://thingsboard.io/blog/a-whole-new-world-of-possibilities-interview-with-edge-architect-part-2/>
29. Top 10 Free & Open-Source IoT Infrastructure Platforms ... - MDCplus,
<https://mdcplus.fi/blog/top-free-iot-platforms-manufacturing-infrastructure/>
30. IoT Study Notes for B.Tech Students | PDF | Internet Of Things - Scribd,
<https://www.scribd.com/document/960641652/IoT-Study-Notes>
31. Full text of "kolban-ESP32" - Internet Archive,
https://archive.org/stream/kolban-ESP32/kolban-ESP32_djvu.txt
32. A Disruption Tolerant Architecture based on MQTT for IoT Applications,
https://www.researchgate.net/publication/318129304_A_Disruption_Tolerant_Architecture_based_on_MQTT_for_IoT_Applications
33. Automation To - Logicbus,
https://www.logicbus.com/assets/PDF/catalogos/ICPDAS-Full_Product_Catalog_2.06.10_en.pdf
34. Data Driven Decision Making with Time Series and Spatio-temporal,
<https://arxiv.org/html/2503.08473v3>
35. Real-Time Anomaly Detection Using Neural Networks and Stream,
https://www.researchgate.net/publication/405515313_Real-Time_Anomaly_Detection_Using_Neural_Networks_and_Stream_Processing
36. Day 35: Data Cleaning and Handling Missing Data - by sysdai,
<https://aieworks.substack.com/p/day-35-data-cleaning-and-handling>
37. Data Cleaning of Data Streams - arXiv, <https://arxiv.org/html/2507.20839v1>

38. A Scalable Big Data Framework for Real-Time ... - Research Square,
https://assets-eu.researchsquare.com/files/rs-7330045/v1_covered_e9a4f3ea-f5ea-44fe-b6c3-4871e5e5d588.pdf
39. A Resource-Efficient Framework for Real-Time Anomaly Detection in,
https://www.sciopen.com/local/article_pdf/10.32604/cmcs.2025.065698.pdf
40. Estimating Missing Data in Data Streams | Request PDF,
https://www.researchgate.net/publication/220788112_Estimating_Missing_Data_in_Data_Streams
41. Stream Processing in IoT Systems | PDF | Internet Of Things - Scribd,
<https://www.scribd.com/presentation/961316461/Iot-Module-2>
42. Real-Time Fraud Detection - Ververica,
<https://www.ververica.com/banking/fraud-detection>
43. Condition Monitoring and Predictive Maintenance with Apache Kafka,
<https://www.kai-waehner.de/blog/2021/10/25/apache-kafka-condition-monitoring-predictive-maintenance-industrial-iot-digital-twin/>
44. Best database for time series data - BeVisible,
<https://bevisible.app/recommendations/best-database-for-time-series-data>
45. InfluxDB 3 vs TimescaleDB vs ClickHouse for IoT Time Series (2026),
<https://iotdigitaltwinplm.com/influxdb-vs-timescaledb-vs-clickhouse-iot-time-series-2026/>
46. PI System, InfluxDB, and TimescaleDB - iFactory AI,
<https://ifactoryapp.com/blog/historian-database-comparison>
47. View of Real-Time Big Data Analytics with Apache Flink and Python,
<https://irjaeh.com/index.php/journal/article/view/1934/1770>
48. developing scalable data pipelines for real-time anomaly detection,
https://www.researchgate.net/publication/393435370_DEVELOPING_SCALABLE_DATA_PIPELINES_FOR_REAL-TIME_ANOMALY_DETECTION_IN_INDUSTRIAL_IOT_SENSOR_NETWORKS
49. Nussknacker: Low-Code Apache Flink, <https://nussknacker.io/>
50. Architecting Real-Time Data Streaming and Analytics Pipelines in,
<https://developersvoice.com/blog/architecture/realtime-data-pipelines-dotnet-kafka-flink/>
51. Anomaly Detection in Industrial IoT Sensor Streams - JETIR.org,
<https://www.jetir.org/papers/JETIR2609032.pdf>
52. A Hybrid and Modular Integration Concept for Anomaly Detection in,
<https://www.mdpi.com/2673-2688/6/5/91>
53. A PoC Approach: Real-Time Fraud Detection with Kafka, Flink & ML,
<https://medium.com/@sugumar.p/a-poc-approach-real-time-fraud-detection-with-kafka-flink-ml-2c7074bb1136>
54. Kafka | Docs | ThingsBoard PE,
<https://thingsboard.io/docs/pe/reference/rule-engine/nodes/external/kafka/>
55. Experimental Test-bed Setup of the Proposed LoRa Based SCADA,
https://www.researchgate.net/figure/Experimental-Test-bed-Setup-of-the-Proposed-LoRa-Based-SCADA-System_fig3_347868716
56. The Real Cost of Open Source SCADA,

- <https://bellascada.com/en/blog/acik-kaynak-scada-gercek-maliyeti>
57. GitHub - Viktor-T/plat-edu-bad-data-mvp,
<https://github.com/Viktar-T/plat-edu-bad-data-mvp>
 58. Simple Scada/Monitor App. : r/PLC - Reddit,
https://www.reddit.com/r/PLC/comments/1rxajlp/simple_scadamonitor_app/
 59. Greenhouse PLC Migration: CODESYS vs Priva, Argus, Hoogendoo,
<https://industrialmonitordirect.com/blogs/knowledgebase/replacing-priva-greenhouse-controls-with-open-codesys-plc-systems>
 60. Best Free SCADA Software in 2026: 12 Open-Source and Free,
<https://scadaprotocols.com/free-scada-software/>
 61. Durable message queues with RabbitMQ · Issue #3042 ... - GitHub,
<https://github.com/thingsboard/thingsboard/issues/3042>
 62. Docker Catalog applications - Hostinger,
<https://www.hostinger.com/support/hostinger-docker-catalog-applications/>
 63. VPS Docker Templates | One-Click App Deployment ... - Hostinger,
<https://www.hostinger.com/applications>
 64. 2.4 Architecture Takeaways and Extensions | 从工业软件到 AI 智能体,
<https://book.dc3.site/en/foundations/chapter-2/2-4>
 65. Deployment Scenarios | Docs | ThingsBoard PE,
<https://thingsboard.io/docs/pe/reference/architecture/deployment-scenarios/>